

Troubleshooting: One Website on Multiple Ports

Issues specific to the multi-port-single-site pattern, beyond the generic “nginx won’t start” basics.

1. Both ports work, but content differs between them

Symptom: `curl :8081` and `curl :8082` return different HTML.

Cause: you have two separate `server { }` blocks with different `root` values instead of one block with two `listen` lines — or a leftover config from an earlier per-port setup is still enabled.

```
sudo nginx -T | grep -A2 "listen 808"
ls /etc/nginx/sites-enabled/      # Ubuntu/Debian
ls /etc/nginx/conf.d/             # CentOS/RHEL
```

Fix: consolidate into a single `server { listen 8081; listen 8082; root /var/www/mysite; ... }` block and remove/disable the duplicate old configs.

2. nginx -t succeeds, but only ONE of the ports responds

```
sudo ss -tulpn | grep nginx
```

If only 8081 shows up: - Check for a typo in the second `listen` line (e.g. `listen 8082` missing the semicolon merges it into the next directive). - Check you didn’t accidentally define `listen 8082` inside a *different, unrelated* `server` block that has `server_name` restrictions preventing it from matching your request. - Confirm you reloaded after editing: `sudo systemctl reload nginx`.

3. X-Served-Port header missing from response

```
curl -I http://localhost:8081
```

- `add_header` directives are not inherited into a `location` block that already has its own `add_header` — if you added more headers inside `location /`, restate all `add_header` lines there, or move them to the server level (as in the README) so they apply everywhere.
 - If a `location ~ /\.` or similar block returns a response (e.g. `deny all` → 403) before reaching your normal `location /`, headers set only in `location /` won’t appear on that response.
-

4. Log file doesn't show which port a request hit

Symptom: `mysite_access.log` looks like a normal combined log with no port info.

Cause: the custom `log_format multiport` was defined but `access_log` still references the default format (or a typo in the format name).

```
sudo grep -n "log_format\|access_log" /etc/nginx/sites-available/mysite
```

Fix: confirm `access_log /var/log/nginx/mysite_access.log multiport;` — the format name must match exactly what's in `log_format multiport '...'`, and `log_format` must be declared in the `http` context (top of `nginx.conf` or an included file), not inside a server block.

5. Health check on 8083 returns 404 instead of "OK"

```
curl -i http://localhost:8083/health
```

Checklist: - File exists: `ls -l /var/www/mysite/health/index.html` - location = `/health` (exact match) requires the URL to be exactly `/health` with no trailing slash — `/health/` won't match an exact-match block. Use location `/health` (prefix match) if you want both to work, or add a second location = `/health/`. - `try_files /health/index.html =503;` — the path here is relative to root, so root must still be `/var/www/mysite` in that server block.

6. Health port (8083) is reachable from the public internet

This defeats the purpose of separating it out.

```
sudo ufw status numbered
sudo firewall-cmd --list-all-zones
```

Fix: scope the firewall rule to your monitoring/internal network only, as shown in the README (`ufw allow from 10.0.0.0/8 to any port 8083`), not a blanket `allow 8083/tcp`. Also double check no cloud security group opens 8083 to `0.0.0.0/0`.

7. Redirect from 8082 → 8081 causes a redirect loop

Symptom: browser shows "too many redirects" when hitting port 8082.

Cause: `$host` in the return 301 doesn't include the port, so if something upstream (a proxy, or the browser itself via a bookmark) keeps re-adding `:8082`, or if 8081 is *also* configured to redirect back,

you get a loop.

```
curl -I http://localhost:8082/
```

Fix: make sure only one direction redirects, and that the target server block (8081) does NOT itself have a return directive redirecting elsewhere.

8. gzip not applying (Content-Encoding missing)

```
curl -I -H "Accept-Encoding: gzip" http://localhost:8081/
```

- Nginx must be compiled with `--with-http_gzip_static_module` or `gzip` built in (check with `nginx -V 2>&1 | grep gzip`).
 - `gzip_min_length 256`; means files smaller than 256 bytes won't be compressed — test with a larger file, not a 20-byte `index.html`.
 - Some file types aren't in your `gzip_types` list — add the MIME type explicitly.
-

9. Changing root doesn't seem to take effect on one of the ports

- If you have `root` set both at the server level and again inside a location block, the location-level `root` wins for that location. Check for stray `root` lines:

```
sudo grep -n "root " /etc/nginx/sites-available/mysite
```

- Always `sudo nginx -t` & `sudo systemctl reload nginx` after any change — nginx does not hot-reload config files on save.
-

Quick Diagnostic Checklist (multi-port specific)

```
sudo nginx -T | less           # 1. Full effective
config – confirms no duplicate/conflicting blocks
sudo ss -tulpn | grep nginx    # 2. Which ports is
nginx actually listening on?
curl -s :8081 | md5sum; curl -s :8082 | md5sum # 3. Are both ports
serving identical content?
curl -I :8081 | grep X-Served-Port # 4. Debug header
present and correct?
sudo tail -20 /var/log/nginx/mysite_access.log # 5. Is
$server_port showing up per line?
curl :8083/health              # 6. Health endpoint
isolated and working?
sudo ufw status / firewall-cmd --list-all # 7. Is the health port
actually restricted?
```

If all seven pass and something's still off, capture `sudo nginx -T` output and the exact `curl -v` output for the failing port — that combination almost always pinpoints which server block is actually answering the

request.